Custom Reinforcement Learning Agent using Farama Gymnasium

COMSATS University, Attock Campus | Topics in AI | Assignment I

Student name: ____maha ishaq_____ Registration no.: _____fa23-bai-037_____

Repository: https://github.com/MahaIshaq04/taxi-q-learning-assignment

1. Problem description

Taxi-v3 models passenger pickup and delivery on a 5 × 5 grid with internal walls and four landmarks. The agent must collect a passenger and deliver them to the requested destination. The environment is initialized with gymnasium.make("Taxi-v3") under Gymnasium 1.2.1, using its default deterministic dynamics.

There are 500 encoded states: 25 taxi positions × 5 passenger locations (four landmarks or inside the taxi) × 4 destinations. The encoding is s = ((row × 5 + column) × 5 + passenger) × 4 + destination. Only a subset is reachable, with 300 possible initial states. Six actions are available: south, north, east, west, pickup, and dropoff.

Event	Reward / episode behavior
Ordinary move or wall collision	−1
Illegal pickup or dropoff	−10
Correct delivery	+20; episode terminates
200-step time limit	Episode truncates

2. Methodology

Tabular Q-learning is implemented from scratch with NumPy. Each Q(s,a) estimates the discounted return for taking action a in state s and then following a greedy policy. The 500 × 6 float64 table starts at zero. After each transition, the agent applies:

$$\mathbf{Q(s,a) \leftarrow Q(s,a) + \alpha\,[r + \gamma(1 - d)\,\max_{\beta} Q(s',b) - Q(s,a)]}$$

Here α is the learning rate, γ is the discount, and d indicates true termination. Successful delivery suppresses bootstrapping. A time-limit truncation ends the rollout but retains bootstrapping because the limit is external to the underlying task.

Training uses epsilon-greedy exploration: a uniformly random action with probability ε, otherwise a maximal-Q action with random tie-breaking. $\varepsilon_e = \max(0.01, 1.0 \times 0.999^e)$, where e starts at zero. Greedy evaluation uses deterministic argmax. No action masking, predefined routes, planning model, or RL framework is used.

Implementation architecture

agent.py owns Q-values, updates, action selection, and persistence. utils.py handles training, evaluation, plots, and GIF rendering. main.py orchestrates experiments. assignment.ipynb displays executed training and evaluation outputs. All dependencies are installed inside the local .venv.

3. Experimental setup and hyperparameters

Parameter	Final selected value
alpha	0.1
gamma	0.99
epsilon_start	1.0
epsilon_min	0.01
epsilon_decay	0.999
episodes	12000
Training seeds	0, 1, 2
Maximum steps / episode	200
Selected model seed	0

The sensitivity study compares $\alpha \in \{0.1, 0.5, 0.9\}$, keeping all other settings fixed: nine runs of 12,000 episodes (108,000 episodes in total). Validation every 1,000 episodes uses reset seeds 20000–20099. The highest mean final validation return across three training seeds selects α; ties select the lower α. Deployment always uses the predeclared seed 0.

After selection, every trained policy and a uniform-random baseline are evaluated on the same 500 separate test resets (30000–30499). Test results do not select hyperparameters. Validation does not update the Q-table or consume the training action RNG. The demonstration uses another reset seed, 40000.

Executed training-progress output

Episode	Greedy validation return	Success (100 resets)
1000	3.44	98.0%
2000	7.54	100.0%
3000	7.54	100.0%
4000	7.54	100.0%
5000	7.54	100.0%
6000	7.54	100.0%
7000	7.54	100.0%
8000	7.54	100.0%
9000	7.54	100.0%
10000	7.54	100.0%
11000	7.54	100.0%
12000	7.54	100.0%

This is the actual checkpoint output from the notebook's fresh α=0.5, seed-0 run. It is a demonstration run, separate from the validation-selected α=0.1 saved model. Training logs preserve every episode for all nine study runs.

Reproducibility outputs

Saved model: artifacts/q_table.npy; shape: (500, 6); 3,000 float64 action values. Raw outputs: nine *_training.csv logs, nine *_validation.csv logs, nine per-run *_q.npy models, and results.json. The notebook contains six successfully executed code cells. Full numerical logs remain in the repository; this report presents their measured summaries.

4. Results and comparative analysis

Policy	Success	Mean return	Mean steps	Illegal / ep.
Selected Q-learning	100% (500/500)	8.030	12.970	0
Uniform random	4.2% (21/500)	−773.024	197.618	64.032

The selected policy (α=0.1, seed 0) completes every sampled test episode without illegal actions. Random behavior frequently incurs pickup/dropoff penalties and reaches the time limit. The selected policy's return standard deviation across test episodes is 2.553, reflecting differences among starting tasks.

All nine trained-policy test outputs

α	Seed	Mean return	Mean steps	Success	Illegal / ep.
0.1	0	8.030	12.970	100%	0
0.1	1	8.030	12.970	100%	0
0.1	2	8.030	12.970	100%	0
0.5	0	7.994	13.006	100%	0
0.5	1	7.966	13.034	100%	0
0.5	2	8.010	12.990	100%	0
0.9	0	8.030	12.970	100%	0
0.9	1	8.022	12.978	100%	0
0.9	2	8.006	12.994	100%	0

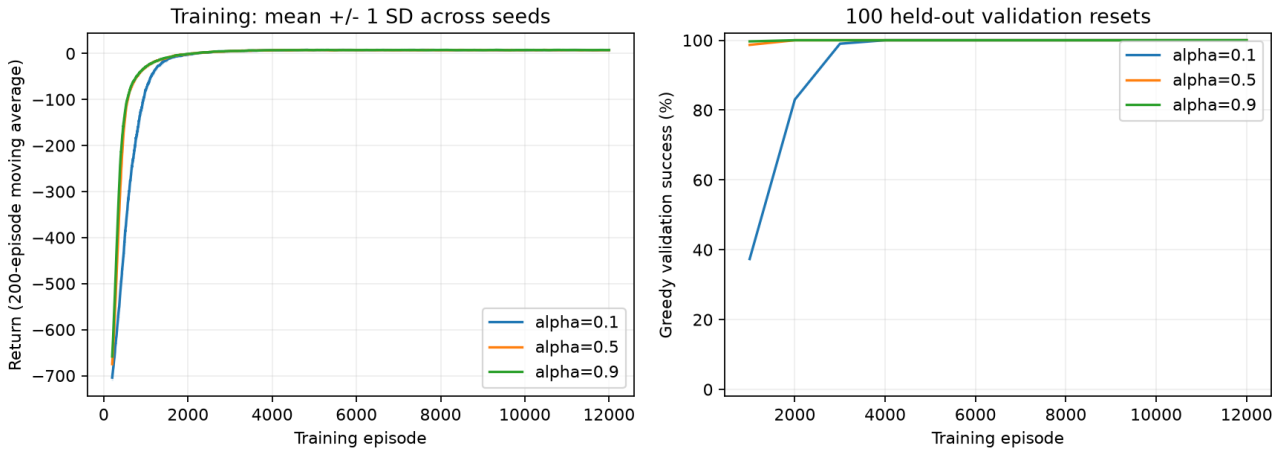


Figure 1. Left: 200-episode moving-average training returns, mean ± one SD over three training seeds. Right: greedy validation success at 1,000-episode checkpoints.

Final test returns (mean ± population SD of seed-level means) are 8.030 ± 0.000 for α=0.1, 7.990 ± 0.018 for α=0.5, and 8.019 ± 0.010 for α=0.9. These descriptive spreads are not confidence intervals. All nine policies reach 100% test success, with small differences in route length.

Higher learning rates improve early learning speed. First observed 100% validation success occurs at episodes 4000/3000/4000 for α=0.1, 2000/2000/1000 for α=0.5, and 2000/1000/1000 for α=0.9 (seeds 0/1/2). The validation-selected α=0.1 learns more slowly but has the best final average validation return. These are observed milestones, not a convergence proof.

5. Visual outputs and learned-policy demonstration

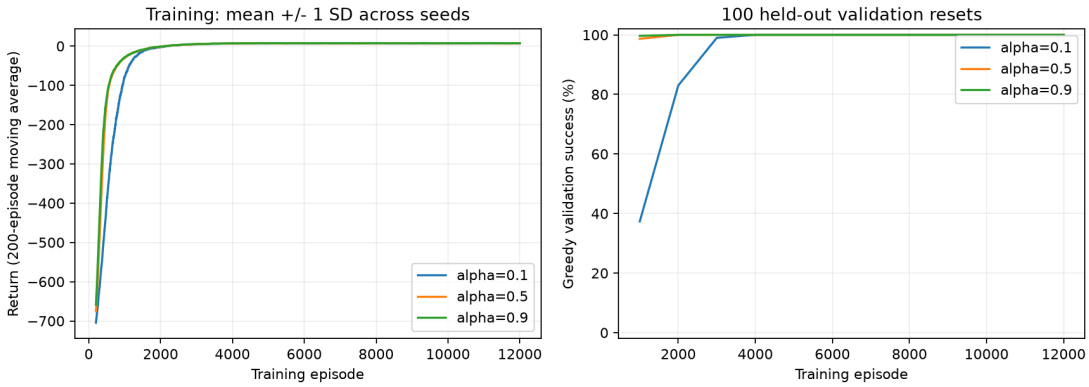


Figure 2. Actual fresh seed-0, α=0.5 training plot from the executed notebook.

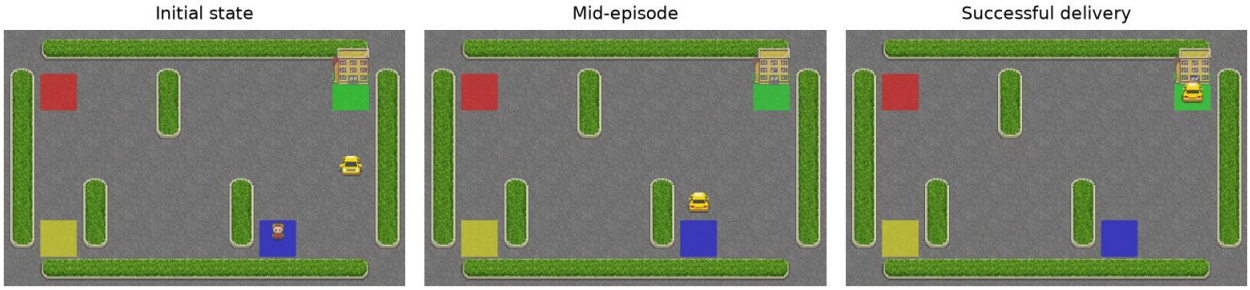


Figure 3. Frames from taxi_policy.gif. Reset seed 40000; successful delivery in 10 steps; total return 11. The GIF and executed notebook contain the full animation.

6. Discussion and failure modes

Early exploration produces illegal pickup/dropoff attempts, wall collisions, and unnecessary movements. Before enough learning, greedy ties or inaccurate values can cause loops until truncation. The ε=0.01 exploration floor continues to introduce occasional training penalties even when the greedy policy is reliable; training returns therefore differ from evaluation returns.

Limitations: only three training seeds and 500 sampled test resets were used, rather than exhaustive verification. Only learning rate was varied. Constant learning rates and a finite budget do not prove convergence. Performance on a small deterministic grid does not establish generalization to changing traffic, stochastic maps, or continuous control.

7. Conclusion

From-scratch Q-learning solves the sampled Taxi-v3 test tasks with 100% delivery success, mean return 8.030, and mean length 12.970 steps. Independent validation/test resets, a random baseline, multiple seeds, saved models, executed plots, and a successful animation support these empirical findings.

References

Farama Foundation, Taxi: https://gymnasium.farama.org/environments/toy_text/taxi/
Farama Foundation, Handling Time Limits: https://gymnasium.farama.org/tutorials/gymnasium_basics/handling_time_limits/
Sutton & Barto (2018), Reinforcement Learning: An Introduction, 2nd ed., Chapter 6.
Environment facts were checked against installed Gymnasium 1.2.1; current web documentation may describe a newer version.